Complete SQL Learning Roadmap

🟢 LEVEL 1 — SQL Fundamentals

1. Database Basics

Understand:

- What is a database?
- DBMS vs RDBMS
- Tables
- Rows
- Columns
- Primary key
- Foreign key
- Constraints
- Relationships
- SQL vs MySQL vs PostgreSQL vs SQL Server vs Oracle

Basic structure:

```
Database
 ├── Table
 │    ├── Column
 │    ├── Column
 │    └── Column
 │
 └── Table
      ├── Column
      └── Column
```

🟢 LEVEL 2 — SELECT Statements

Learn these first.

2. SELECT

```sql
SELECT * FROM students;
```

```sql
SELECT student_name, marks
FROM students;
```

3. DISTINCT

```sql
SELECT DISTINCT class
FROM students;
```

4. Column aliases

```sql
SELECT
    student_name AS name,
    marks AS score
FROM students;
```

5. Expressions

```sql
SELECT
    student_name,
    marks,
    marks + 5 AS new_marks
FROM students;
```

🟢 LEVEL 3 — Filtering Data

6. WHERE

```sql
SELECT *
FROM students
WHERE marks > 70;
```

7. Comparison operators

Learn:

```
=
<>
!=
```

```
>
<
>=
<=
```

8. AND

```sql
SELECT *
FROM students
WHERE marks > 70
AND class = 10;
```

9. OR

```sql
SELECT *
FROM students
WHERE class = 9
OR class = 10;
```

10. NOT

```sql
SELECT *
FROM students
WHERE NOT gender = 'Girl';
```

11. BETWEEN

```sql
SELECT *
FROM students
WHERE marks BETWEEN 60 AND 80;
```

12. IN

```sql
SELECT *
FROM students
WHERE class IN (8, 9, 10);
```

13. NOT IN

```sql
SELECT *
FROM students
WHERE class NOT IN (8, 9);
```

14. LIKE

```sql
SELECT *
FROM students
WHERE student_name LIKE 'A%';
```

Learn patterns:

```
'A%'     → starts with A
'%A'     → ends with A
'%A%'    → contains A
'_A%'    → second character is A
```

15. IS NULL

```sql
SELECT *
FROM students
WHERE phone IS NULL;
```

16. IS NOT NULL

```sql
SELECT *
FROM students
WHERE phone IS NOT NULL;
```

🟢 LEVEL 4 — Sorting and Limiting

17. ORDER BY

```sql
SELECT *
FROM students
ORDER BY marks;
```

18. DESC

```
SELECT *
FROM students
ORDER BY marks DESC;
```

19. Multiple columns

```
SELECT *
FROM students
ORDER BY class ASC, marks DESC;
```

20. LIMIT

```
SELECT *
FROM students
LIMIT 10;
```

Also learn database-specific alternatives:

```
LIMIT
TOP
FETCH FIRST
OFFSET
```

🟡 LEVEL 5 — Aggregate Functions

Learn:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

COUNT

```sql
SELECT COUNT(*)
FROM students;
```

SUM

```sql
SELECT SUM(marks)
FROM students;
```

AVG

```sql
SELECT AVG(marks)
FROM students;
```

MIN

```sql
SELECT MIN(marks)
FROM students;
```

MAX

```sql
SELECT MAX(marks)
FROM students;
```

🟡 LEVEL 6 — GROUP BY

This is one of the most important SQL topics.

Basic GROUP BY

```sql
SELECT class, COUNT(*)
FROM students
GROUP BY class;
```

Multiple columns

```
SELECT class, gender, COUNT(*)
FROM students
GROUP BY class, gender;
```

GROUP BY + SUM

```
SELECT class, SUM(marks)
FROM students
GROUP BY class;
```

GROUP BY + AVG

```
SELECT class, AVG(marks)
FROM students
GROUP BY class;
```

🟡 LEVEL 7 — HAVING

Understand the difference:

```
WHERE  → filters rows
HAVING → filters groups
```

Example:

```
SELECT class, AVG(marks) AS average_marks
FROM students
GROUP BY class
HAVING AVG(marks) > 70;
```

🟡 LEVEL 8 — CASE WHEN

Learn conditional logic.

Basic CASE

```
SELECT
    student_name,
    marks,
    CASE
        WHEN marks >= 90 THEN 'A'
        WHEN marks >= 75 THEN 'B'
        WHEN marks >= 60 THEN 'C'
        ELSE 'D'
    END AS grade
FROM students;
```

CASE with multiple conditions

```
CASE
    WHEN marks >= 90 THEN 'Excellent'
    WHEN marks >= 75 THEN 'Good'
    WHEN marks >= 50 THEN 'Average'
    ELSE 'Fail'
END
```

🟡 LEVEL 9 — Conditional Aggregation

You are currently learning this topic.

Count boys and girls

```
SELECT
    class,
    SUM(CASE WHEN gender = 'Boy' THEN 1 ELSE 0 END) AS boys,
    SUM(CASE WHEN gender = 'Girl' THEN 1 ELSE 0 END) AS girls,
    COUNT(*) AS total
FROM students
GROUP BY class;
```

Also learn:

```
COUNT(CASE WHEN gender = 'Boy' THEN 1 END)
```

and, depending on the database:

```
COUNT(*) FILTER (WHERE gender = 'Boy')
```

🟠 LEVEL 10 — JOINS

This is one of the most important SQL sections.

Suppose you have:

```
students
classes
teachers
departments
```

Learn:

1. INNER JOIN

```
SELECT *
FROM students s
INNER JOIN classes c
    ON s.class = c.class;
```

2. LEFT JOIN

```
SELECT *
FROM students s
LEFT JOIN classes c
    ON s.class = c.class;
```

3. RIGHT JOIN

```
SELECT *
FROM students s
RIGHT JOIN classes c
    ON s.class = c.class;
```

4. FULL OUTER JOIN

```
SELECT *
FROM students s
FULL OUTER JOIN classes c
    ON s.class = c.class;
```

5. CROSS JOIN

```
SELECT *
FROM students
CROSS JOIN subjects;
```

6. SELF JOIN

Joining a table to itself:

```
SELECT
    e.employee_name,
    m.employee_name AS manager
FROM employees e
JOIN employees m
    ON e.manager_id = m.employee_id;
```

🟠 LEVEL 11 — UNION

Learn:

UNION

```
SELECT student_name FROM class_10
UNION
SELECT student_name FROM class_9;
```

Removes duplicates.

UNION ALL

```
SELECT student_name FROM class_10
UNION ALL
SELECT student_name FROM class_9;
```

Keeps duplicates.

Also learn:

```
INTERSECT
EXCEPT
```

Availability varies by database.

🟠 LEVEL 12 — String Functions

Learn common functions:

```
CONCAT()
LOWER()
UPPER()
LENGTH()
TRIM()
LTRIM()
RTRIM()
SUBSTRING()
REPLACE()
LEFT()
RIGHT()
POSITION()
```

Example:

```
SELECT
    UPPER(student_name) AS name
FROM students;
```

🟠 LEVEL 13 — Numeric Functions

Learn:

```
ROUND()
CEILING()
FLOOR()
ABS()
MOD()
POWER()
SQRT()
```

Example:

```
SELECT ROUND(AVG(marks), 2)
FROM students;
```

🟠 LEVEL 14 — Date and Time Functions

Very important for real-world SQL.

Learn:

```
CURRENT_DATE
CURRENT_TIMESTAMP
EXTRACT()
DATE_PART()
DATE_TRUNC()
DATEDIFF()
DATE_ADD()
DATE_SUB()
```

Exact functions differ between MySQL, PostgreSQL, SQL Server, etc.

Example:

```sql
SELECT
    EXTRACT(YEAR FROM admission_date) AS admission_year
FROM students;
```

🟠 LEVEL 15 — NULL Handling

Learn:

COALESCE

```sql
SELECT
    student_name,
    COALESCE(phone, 'Not Available') AS phone
FROM students;
```

NULLIF

```sql
SELECT NULLIF(marks, 0)
FROM students;
```

Understand:

```
NULL
0
''
'NULL'
```

They are NOT the same thing.

🔵 LEVEL 16 — Subqueries

Learn subqueries carefully.

Scalar subquery

```sql
SELECT student_name, marks
FROM students
```

```
WHERE marks > (
    SELECT AVG(marks)
    FROM students
);
```

IN subquery

```
SELECT *
FROM students
WHERE class IN (
    SELECT class
    FROM classes
    WHERE section = 'A'
);
```

EXISTS

```
SELECT *
FROM students s
WHERE EXISTS (
    SELECT 1
    FROM fees f
    WHERE f.student_id = s.student_id
);
```

Correlated subquery

```
SELECT *
FROM students s
WHERE marks > (
    SELECT AVG(marks)
    FROM students s2
    WHERE s2.class = s.class
);
```

🔵 LEVEL 17 — CTEs

CTE = Common Table Expression.

Basic CTE

```sql
WITH class_average AS (
    SELECT
        class,
        AVG(marks) AS avg_marks
    FROM students
    GROUP BY class
)
SELECT *
FROM class_average;
```

Multiple CTEs

```sql
WITH class_average AS (
    SELECT class, AVG(marks) AS avg_marks
    FROM students
    GROUP BY class
),
top_classes AS (
    SELECT *
    FROM class_average
    WHERE avg_marks > 70
)
SELECT *
FROM top_classes;
```

Recursive CTE

Later learn recursive queries for:

```
Employee hierarchy
Organization structure
Category trees
Parent-child relationships
Graph-like data
```

🔵 LEVEL 18 — Window Functions ⭐

This is a major advanced SQL topic.

Understand:

```
GROUP BY
vs
WINDOW FUNCTIONS
```

`GROUP BY` reduces rows.

Window functions keep the rows and calculate across them.

ROW_NUMBER

```sql
SELECT
    student_name,
    class,
    marks,
    ROW_NUMBER() OVER (
        PARTITION BY class
        ORDER BY marks DESC
    ) AS row_num
FROM students;
```

RANK

```sql
RANK() OVER (
    PARTITION BY class
    ORDER BY marks DESC
)
```

DENSE_RANK

```sql
DENSE_RANK() OVER (
    PARTITION BY class
    ORDER BY marks DESC
)
```

Understand the difference:

```
ROW_NUMBER()
RANK()
DENSE_RANK()
```

🔵 LEVEL 19 — Advanced Window Functions

Learn:

```
LAG()
LEAD()
FIRST_VALUE()
LAST_VALUE()
NTH_VALUE()
NTILE()
```

Example:

```
SELECT
    student_name,
    marks,
    LAG(marks) OVER (
        ORDER BY student_id
    ) AS previous_marks
FROM students;
```

Running total

```
SELECT
    student_id,
    fee,
    SUM(fee) OVER (
        ORDER BY student_id
    ) AS running_total
FROM fees;
```

Partitioned aggregate

```
SELECT
    student_name,
    class,
    marks,
    AVG(marks) OVER (
        PARTITION BY class
```

```
    ) AS class_average
FROM students;
```

🔵 LEVEL 20 — DDL: Creating Database Objects

DDL = Data Definition Language.

Learn:

```
CREATE
ALTER
DROP
TRUNCATE
```

CREATE TABLE

```
CREATE TABLE students (
    student_id INT,
    student_name VARCHAR(100),
    class INT,
    gender VARCHAR(10),
    marks INT
);
```

ALTER TABLE

```
ALTER TABLE students
ADD email VARCHAR(100);
```

DROP

```
DROP TABLE students;
```

TRUNCATE

```
TRUNCATE TABLE students;
```

Understand carefully:

```
DELETE
TRUNCATE
DROP
```

They are different.

🔵 LEVEL 21 — Constraints

Learn:

```
PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
DEFAULT
```

Example:

```sql
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    marks INT CHECK (marks >= 0),
    status VARCHAR(20) DEFAULT 'Active'
);
```

🔵 LEVEL 22 — DML

DML = Data Manipulation Language.

Learn:

```
INSERT
UPDATE
DELETE
```

INSERT

```
INSERT INTO students
(student_id, student_name, class)
VALUES
(1, 'Aarav', 10);
```

UPDATE

```
UPDATE students
SET marks = 85
WHERE student_id = 1;
```

DELETE

```
DELETE FROM students
WHERE student_id = 1;
```

Always understand why `WHERE` is important.

🔵 LEVEL 23 — Transactions

Learn:

```
BEGIN
COMMIT
ROLLBACK
SAVEPOINT
```

Example:

```
BEGIN;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2;
```

```
COMMIT;
```

If something goes wrong:

```
ROLLBACK;
```

Learn the **ACID properties**:

```
Atomicity
Consistency
Isolation
Durability
```

🔴 LEVEL 24 — Views

A view is a saved query.

```
CREATE VIEW class_summary AS
SELECT
    class,
    COUNT(*) AS total_students,
    AVG(marks) AS average_marks
FROM students
GROUP BY class;
```

Then:

```
SELECT *
FROM class_summary;
```

Learn:

```
VIEW
MATERIALIZED VIEW
```

Materialized views depend on database system.

🔴 LEVEL 25 — Indexes

Indexes are extremely important for performance.

Learn:

```
CREATE INDEX idx_student_class
ON students(class);
```

Unique index:

```
CREATE UNIQUE INDEX idx_student_email
ON students(email);
```

Understand:

```
Index
Composite index
Unique index
Covering index
Clustered index
Non-clustered index
```

Some index concepts are database-specific.

🔴 LEVEL 26 — Query Optimization

Learn how SQL actually executes a query.

Understand:

```
Execution Plan
Full Table Scan
Index Scan
Index Seek
Join Algorithms
Sort
Hash
Aggregation
```

Use:

```
EXPLAIN
```

or database-specific variants such as:

```
EXPLAIN ANALYZE
```

Learn why this query may be slow:

```
SELECT *
FROM students
WHERE YEAR(admission_date) = 2026;
```

And why rewriting it can sometimes allow better index usage:

```
SELECT *
FROM students
WHERE admission_date >= '2026-01-01'
  AND admission_date < '2027-01-01';
```

🔴 LEVEL 27 — Database Design

Now learn how to design databases properly.

Relationships

```
One-to-One
One-to-Many
Many-to-Many
```

Normalization

Learn:

```
1NF
2NF
```

```
3NF
BCNF
```

Also understand when denormalization is useful.

Example:

Instead of:

```
students
student_name
class_name
teacher_name
teacher_phone
```

design related tables:

```
students
classes
teachers
```

and connect them using keys.

🔴 LEVEL 28 — Advanced SQL Programming

Depending on your database, learn:

Stored Procedures

```
CREATE PROCEDURE ...
```

User-defined functions

```
CREATE FUNCTION ...
```

Triggers

```
CREATE TRIGGER ...
```

Cursors

```
DECLARE CURSOR
OPEN
FETCH
CLOSE
```

These are more database-specific and are generally less important for analytics than joins, CTEs, and window functions.

🔴 LEVEL 29 — Advanced Query Techniques

Learn:

```
PIVOT
UNPIVOT
ROLLUP
CUBE
GROUPING SETS
```

For example:

```
GROUP BY ROLLUP(class, gender)
```

Also learn:

```
Conditional aggregation
Multiple-level aggregation
Top-N queries
Gaps and islands
Sessionization
Deduplication
Recursive queries
Hierarchical queries
```

🔴 LEVEL 30 — Advanced Data Analysis SQL

These are extremely useful for Data Analyst / Data Scientist / BI roles.

Learn how to solve:

Top N per group

```
ROW_NUMBER()
RANK()
DENSE_RANK()
```

Running totals

```
SUM() OVER()
```

Moving averages

```
AVG() OVER()
```

Previous/next record

```
LAG()
LEAD()
```

Percentage of total

```
SUM(sales) / SUM(SUM(sales)) OVER ()
```

Year-over-year growth

```
LAG()
```

Month-over-month growth

```
LAG()
```

Duplicate detection

```
COUNT(*) OVER()
```

Deduplication

```
ROW_NUMBER() OVER(...)
```

First/last purchase

```
FIRST_VALUE()
LAST_VALUE()
```

🔴 LEVEL 31 — SQL Security

Learn:

```
Users
Roles
Privileges
GRANT
REVOKE
```

Example:

```
GRANT SELECT
ON students
TO analyst;
```

Understand:

```
Authentication
Authorization
Least privilege
SQL injection
Parameterized queries
```

🔴 LEVEL 32 — SQL and Application Development

Learn how SQL connects with programming languages:

```
Python
Java
JavaScript
C#
PHP
```

For example:

```
Python
   ↓
SQL Query
   ↓
Database
   ↓
Result
   ↓
Pandas / Application
```

If you are learning SQL for data analytics, **Python + SQL + Pandas** is a very useful combination.

🟣 LEVEL 33 — Database-Specific SQL

After learning standard SQL, choose one database deeply.

Good choices:

```
PostgreSQL
MySQL
SQL Server
Oracle
```

For analytics, PostgreSQL is an excellent choice.

Learn its specific features after you understand standard SQL.

🟣 LEVEL 34 — Real-World SQL Projects

Don't learn only commands. Build projects.

Project 1 — School Database

Tables:

```
students
teachers
classes
subjects
marks
attendance
fees
```

Practice:

```
Class-wise students
Boys vs girls
Average marks
Top students
Attendance percentage
Fee collection
Teacher assignments
```

Project 2 — E-commerce

Tables:

```
customers
orders
products
order_items
payments
```

Questions:

```
Top customers
Top products
Monthly sales
Average order value
Repeat customers
Revenue by category
```

Project 3 — Banking

Tables:

```
customers
accounts
transactions
branches
```

Practice:

```
Account balances
Transaction history
Monthly deposits
Withdrawals
Top customers
Fraud-like patterns
```

Project 4 — Employee Database

Tables:

```
employees
departments
salaries
managers
```

Practice:

```
Highest salary
Department average
Employee ranking
Manager hierarchy
Salary growth
Top 3 employees per department
```

⭐ Complete SQL Command Checklist

Querying

```
SELECT
DISTINCT
FROM
WHERE
ORDER BY
LIMIT
OFFSET
```

Conditions

```
AND
OR
NOT
IN
NOT IN
BETWEEN
LIKE
IS NULL
IS NOT NULL
```

Aggregation

```
COUNT
SUM
AVG
MIN
MAX
GROUP BY
HAVING
```

Conditional Logic

```
CASE
WHEN
THEN
ELSE
END
```

```
COALESCE
NULLIF
```

Joins

```
INNER JOIN
LEFT JOIN
RIGHT JOIN
FULL OUTER JOIN
CROSS JOIN
SELF JOIN
```

Set Operations

```
UNION
UNION ALL
INTERSECT
EXCEPT
```

Subqueries

```
IN
EXISTS
NOT EXISTS
Correlated Subquery
Scalar Subquery
```

CTE

```
WITH
WITH RECURSIVE
```

Window Functions

```
OVER
PARTITION BY
ROW_NUMBER
RANK
DENSE_RANK
```

```
NTILE
LAG
LEAD
FIRST_VALUE
LAST_VALUE
NTH_VALUE
```

DDL

```
CREATE
ALTER
DROP
TRUNCATE
```

DML

```
INSERT
UPDATE
DELETE
```

Constraints

```
PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
DEFAULT
```

Transactions

```
BEGIN
COMMIT
ROLLBACK
SAVEPOINT
```

Database Objects

```
VIEW
MATERIALIZED VIEW
INDEX
SEQUENCE
```

Advanced

```
PIVOT
UNPIVOT
ROLLUP
CUBE
GROUPING SETS
```

Security

```
GRANT
REVOKE
ROLE
```

Performance

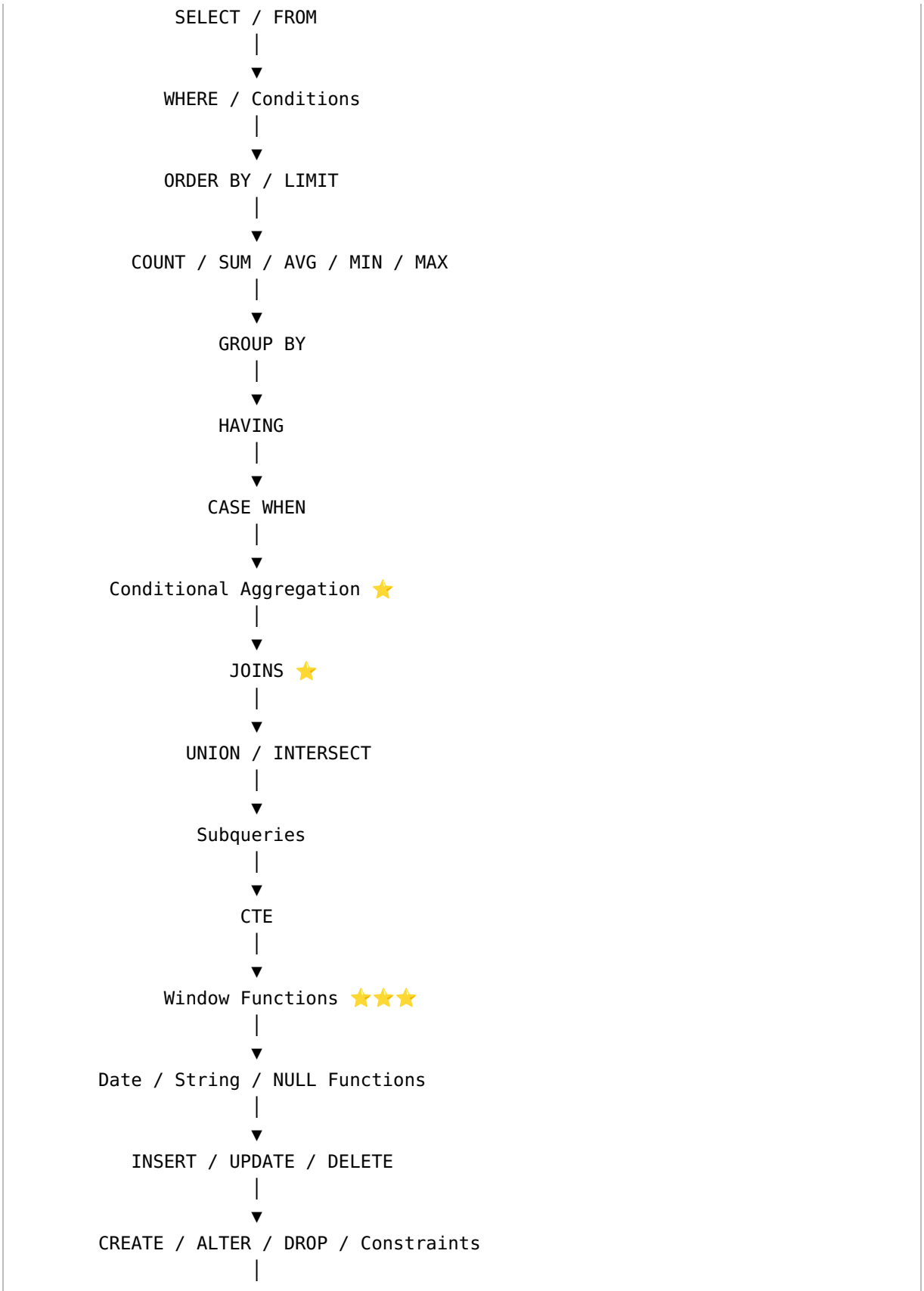
```
EXPLAIN
EXPLAIN ANALYZE
INDEXING
QUERY OPTIMIZATION
EXECUTION PLANS
```

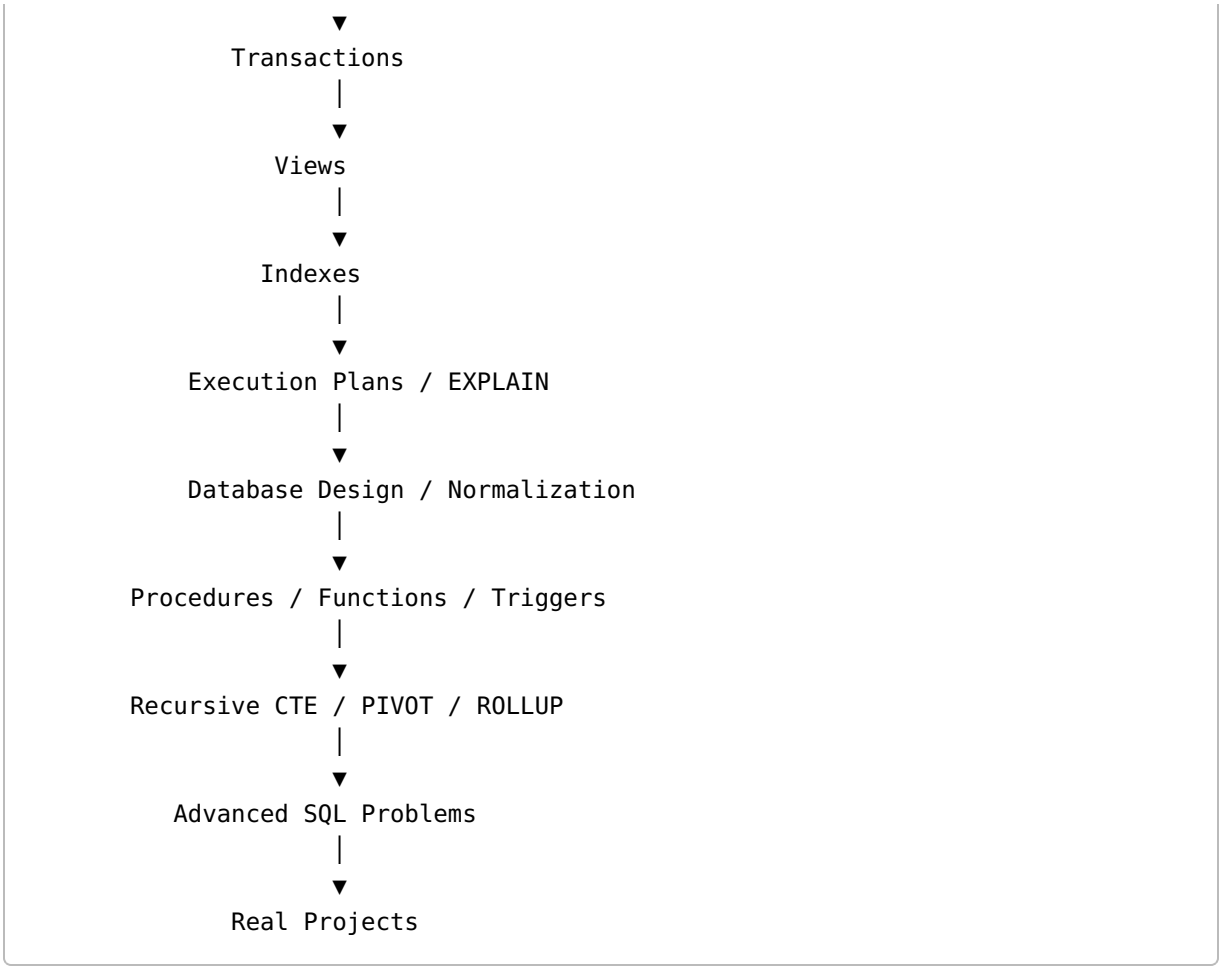
🎯 Recommended Order for You

Don't try to learn all 34 levels simultaneously.

Follow this exact progression:

```
                    SQL
                     |
                     ▼
```


SELECT / FROM
WHERE / Conditions
ORDER BY / LIMIT
COUNT / SUM / AVG / MIN / MAX
GROUP BY
HAVING
CASE WHEN
Conditional Aggregation ⭐
JOINS ⭐
UNION / INTERSECT
Subqueries
CTE
Window Functions ⭐⭐⭐
Date / String / NULL Functions
INSERT / UPDATE / DELETE
CREATE / ALTER / DROP / Constraints



⭐ Most important topics for jobs

If you're learning SQL for **Data Analyst / BI / Data Science**, prioritize these:

```
1. SELECT
2. WHERE
3. ORDER BY
4. GROUP BY
5. Aggregate Functions
6. HAVING
7. CASE WHEN
8. Conditional Aggregation
9. JOINs ⭐⭐⭐
10. Subqueries
11. CTEs ⭐⭐
12. Window Functions ⭐⭐⭐
13. Date Functions
14. String Functions
15. NULL Handling
```

```
16. UNION
17. Real-world SQL problems
18. Query optimization basics
```

You don't need to master stored procedures, triggers, and cursors before becoming good at analytical SQL. **JOINs, CTEs, window functions, aggregation, and solving real business questions are the highest-value skills.**